

多媒体第二次实验

1. 完成本次实验任务的思路

本次实验涵盖了图像、音频和文本三种媒体类型的基本处理，具体思路如下：

- 图像拼贴任务：

- 使用 `cv2.imread` 读取四张猫的图片。
- 为了确保拼贴整齐，利用 `cv2.resize` 将四张图片统一缩放至相同的尺寸（如 500×500 像素）。
- 利用 `numpy` 的矩阵拼接功能（如 `np.hstack` 和 `np.vstack`）将图像排列成 2×2 的矩阵。
- 使用 `cv2.imwrite` 保存生成的拼贴画。

- 音频剪辑任务：

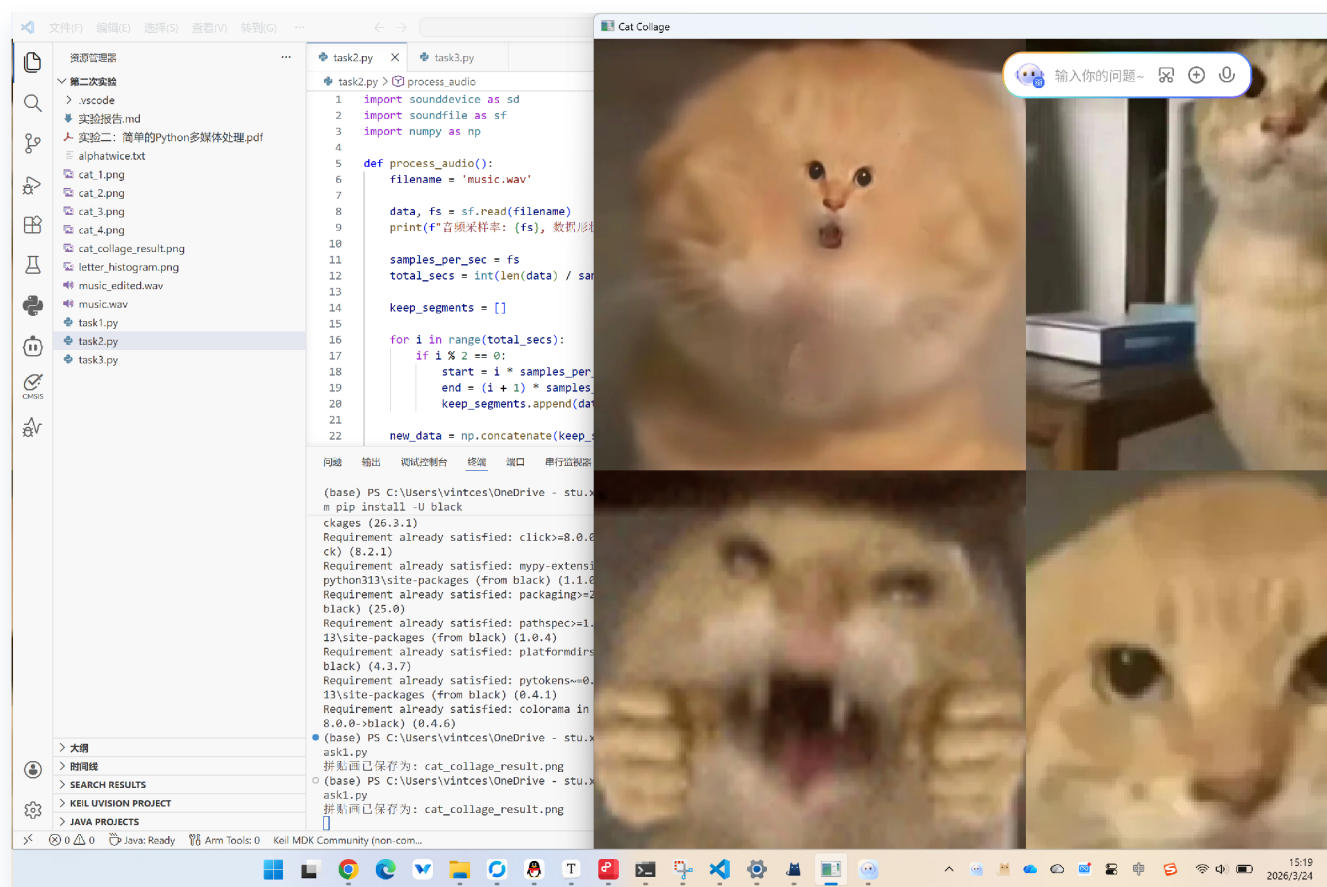
- 使用 `pydub.AudioSegment` 读取 `music.wav` 文件，该库在处理时间切片（毫秒级）上更为直观。
- 通过循环遍历音频时长，以 **1000ms（1秒）** 为步长提取音频片段。
- **逻辑处理**：仅保留奇数秒片段（0-1s, 2-3s...），跳过题目要求的第 2, 4, 6... 等偶数秒。
- 将保留的片段通过 `+` 运算符拼接，并导出为新文件。

- 文本分析与可视化任务：

- 使用 `open()` 函数读取 `alphatwice.txt`。
- 利用字典或 `collections.Counter` 统计英文字母（A-Z, a-z）的出现频率，通过 `isalpha()` 过滤非字母字符。
- 对统计结果进行降序排序，提取前 10 个高频字母。
- 使用 `matplotlib.pyplot` 绘制直方图，设置横轴为字母，纵轴为次数，并调用 `savefig` 保存。

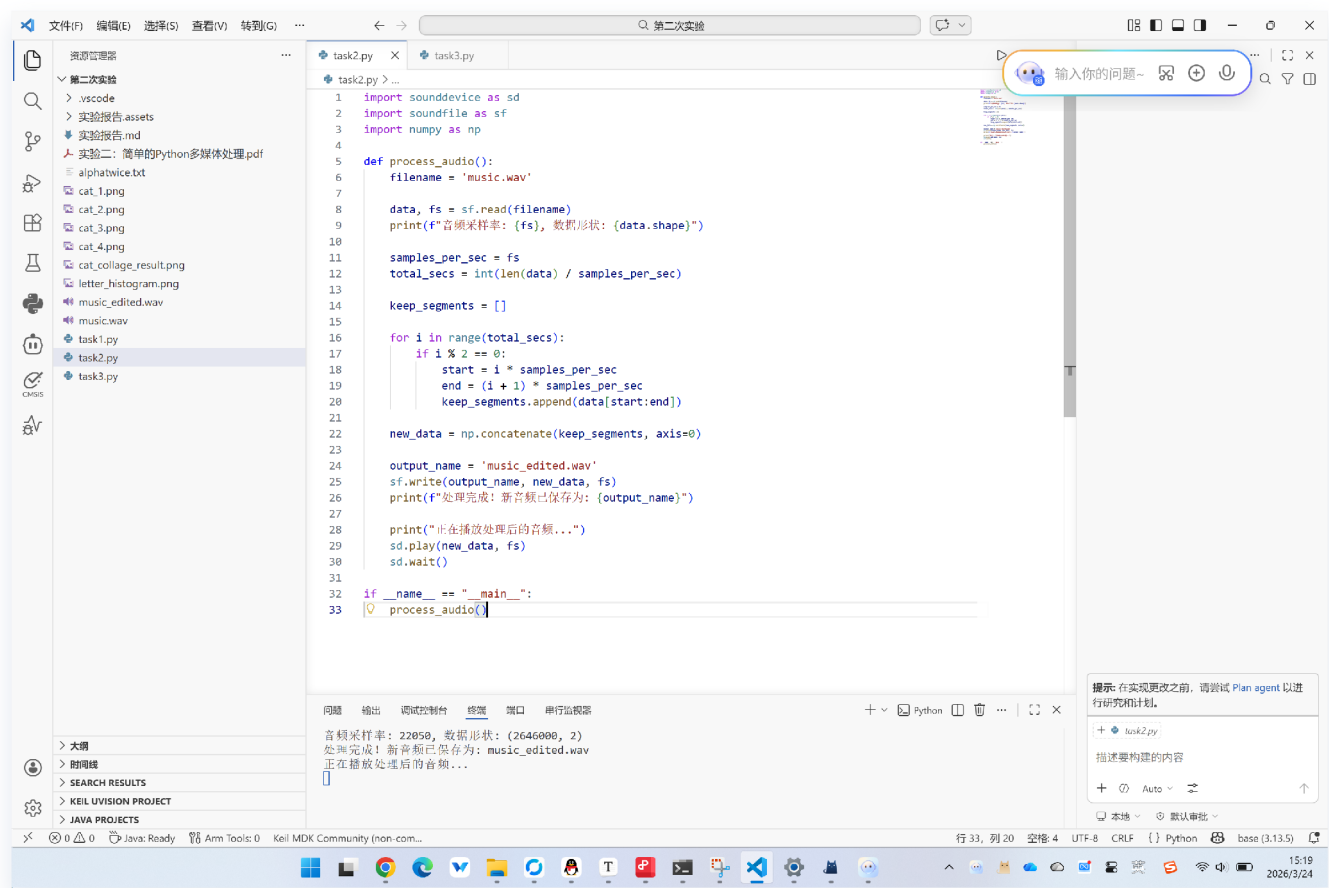
2. 运行程序截图和简要说明

2.1 图像拼贴效果



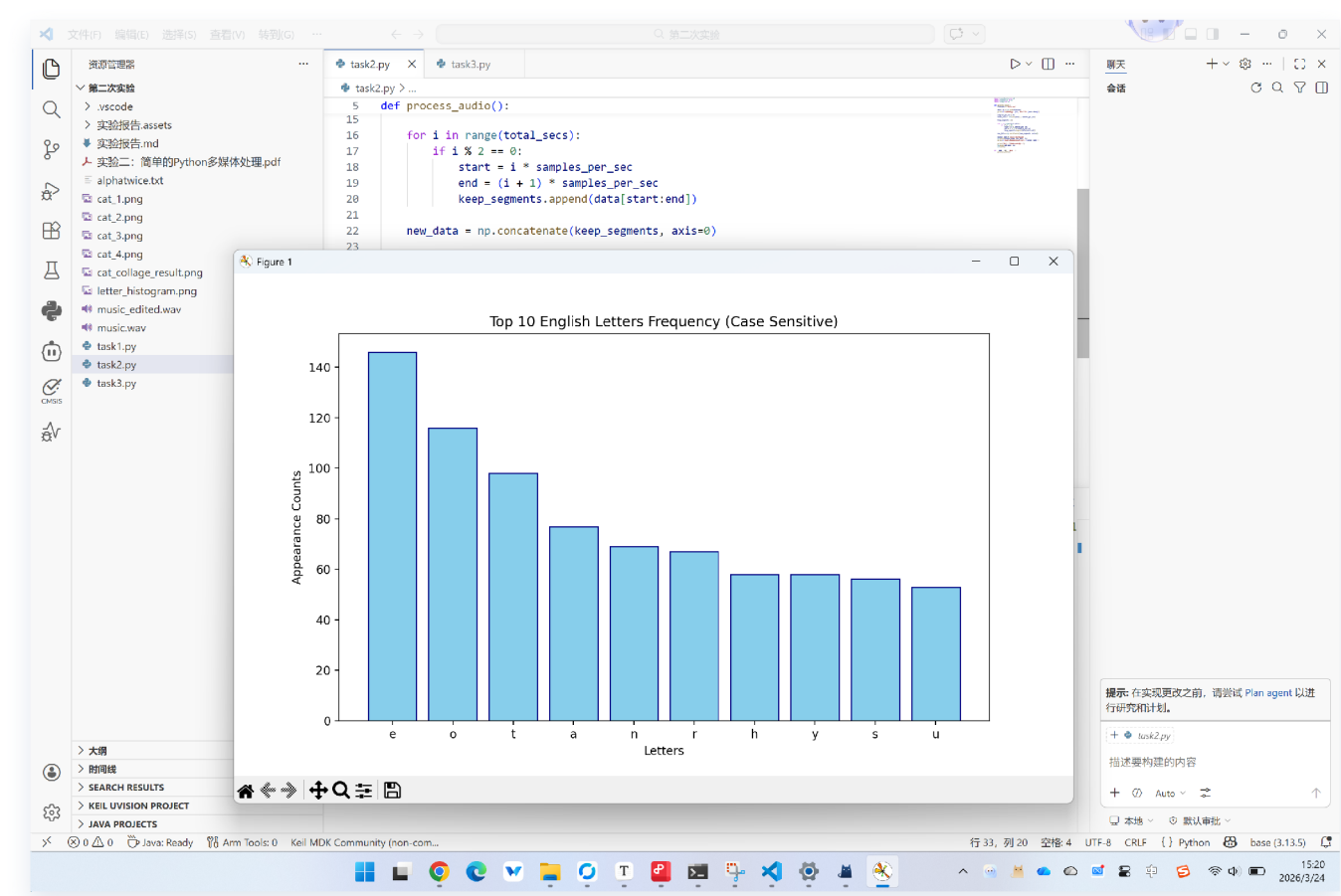
说明：程序读取了四张猫咪图片，利用矩阵拼接技术将其无缝组合成一张大图。

2.2 音频处理状态



说明：处理后的音频成功剔除了偶数秒内容，并通过 `sounddevice` 进行了实时播放验证。

2.3 字母频率直方图



说明：直方图直观地展示了文本中出现频率前 10 的字母及其具体次数。

3. 核心代码展示和分析

3.1 图像拼贴

```
import cv2
import numpy as np

def create_cat_collage():
    img1 = cv2.imread('cat_1.png')
    img2 = cv2.imread('cat_2.png')
    img3 = cv2.imread('cat_3.png')
    img4 = cv2.imread('cat_4.png')

    images = [img1, img2, img3, img4]

    top_row = np.hstack((img1, img2))
    bottom_row = np.hstack((img3, img4))
```

```

collage = np.vstack((top_row, bottom_row))

cv2.imwrite('cat_collage_result.png', collage)
print("拼贴画已保存为: cat_collage_result.png")

cv2.imshow('Cat Collage', collage)
cv2.waitKey(0)
cv2.destroyAllWindows()

if __name__ == "__main__":
    create_cat_collage()

```

代码分析：本段代码的核心在于利用 `numpy.hstack` 实现水平方向的像素矩阵合并，再通过 `numpy.vstack` 将两个水平行进行垂直堆叠，从而在不破坏图像通道数据的前提下完成 2×2 的网格布局构建。

3.2 音频偶数秒剔除

```

import sounddevice as sd
import soundfile as sf
import numpy as np

def process_audio():
    filename = 'music.wav'

    data, fs = sf.read(filename)
    print(f"音频采样率: {fs}, 数据形状: {data.shape}")

    samples_per_sec = fs
    total_secs = int(len(data) / samples_per_sec)

    keep_segments = []

    for i in range(total_secs):
        if i % 2 == 0:
            start = i * samples_per_sec
            end = (i + 1) * samples_per_sec
            keep_segments.append(data[start:end])

    new_data = np.concatenate(keep_segments, axis=0)

    output_name = 'music_edited.wav'
    sf.write(output_name, new_data, fs)

```

```

print(f"处理完成! 新音频已保存为: {output_name}")

print("正在播放处理后的音频...")
sd.play(new_data, fs)
sd.wait()

if __name__ == "__main__":
    process_audio()

```

代码分析：考虑到 Python 3.13 的环境兼容性，代码放弃了传统的 `pydub` 对象处理，直接操作由 `s` `f.read` 返回的 `numpy` 采样数组。通过 `i % 2 == 0` 的逻辑判断保留奇数秒索引，并利用 `np.concatenate` 在时域上对信号进行重组，实现了高效的音频剪辑。

3.3 文本统计与直方图

```

import matplotlib.pyplot as plt
from collections import Counter

def analyze_text():
    with open('alphatwice.txt', 'r', encoding='utf-8') as f:
        content = f.read()

    letters = [char for char in content if char.isalpha()]
    counts = Counter(letters)

    top_10 = counts.most_common(10)

    labels, values = zip(*top_10) if top_10 else ([], [])

    plt.figure(figsize=(10, 6))
    plt.bar(labels, values, color='skyblue', edgecolor='navy')

    plt.title('Top 10 English Letters Frequency (Case Sensitive)')
    plt.xlabel('Letters')
    plt.ylabel('Appearance Counts')

    plt.savefig('letter_histogram.png')
    print("直方图已保存为: letter_histogram.png")
    plt.show()

if __name__ == "__main__":
    analyze_text()

```

代码分析：代码利用了 Python 内置的 `collections.Counter` 类进行高效的频率映射统计，并配合 `str.isalpha()` 过滤掉非字母字符以满足实验要求的统计口径。绘图部分通过 `matplotlib.pyplot.bar` 将统计结果转换为可视化图表，实现了从原始文本到直观数据的转换。